LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Application: *Spam Scoring Filter*
Programming Language: *Python*
Testing Tools: *Hypothesis and pytest*
SRN*:PES1UG24CS582*

1. Objective

The objective of this lab is to understand and implement fuzz testing using the Hypothesis library in Python. Fuzz testing is used to identify bugs by providing random, unexpected, and invalid inputs to a program. It helps detect errors that may not be discovered through normal testing.In this lab, fuzz testing is performed on three functions of the Spam Scoring Filter application to identify input validation problems and fix them.

2. Tools and Technologies Used

- Python 3.11.9
- Hypothesis
- pytest
- Command Prompt (CMD)

3. Functions Tested

The Spam Scoring Filter application contains three main functions:

1. sender_domain()

Extracts the domain name from a sender's email address and converts it into lowercase.

2. first_link()

Finds and extracts the first HTTP link from a given message.

3. shout_ratio()

Calculates the ratio of uppercase alphabetic characters to the total number of alphabetic characters in a message.

4. Implementation of Fuzz Testing

The Hypothesis library was used to generate random inputs automatically.

The following strategy was used:

st.text() | st.none()

This generates text inputs as well as None values to test how the functions handle unexpected inputs.

Three test functions were created:

- test_sender_domain_no_crash()
- test_first_link_no_crash()
- test_shout_ratio_no_crash()

The objective was to ensure that none of the three functions crashed when unexpected inputs were provided.

5. Execution of the Original Application

The original application was executed using:

python spam_scoring.py

Sample inputs were provided for the sender email, message containing a link, and uppercase message.

```
C:\Users\Kiran Nidhi\SE\L6_L7_33_spam_scoring_assigned\lab7>python spam_scoring.py
==== Running sender_domain ====
Enter a sender email (e.g. offers@Deals.com): offers@Deals.com
Sender Domain: deals.com

==== Running first_link ====
Enter a message containing a link: Congratulations! Visit https://example.com now
First Link: https://example.com

==== Running shout_ratio ====
Enter a message (e.g. FREE prize NOW): FREE prize NOW
Uppercase Ratio: 0.5833333333333334

C:\Users\Kiran Nidhi\SE\L6_L7_33_spam_scoring_assigned\lab7>
```

6. Identification of Bugs Using Fuzz Testing

The initial fuzz tests were executed using:

python -m pytest -v test_spam_scoring.py

Hypothesis identified the following problems:

- **sender_domain():** Produced an IndexError for empty input and an AttributeError for None input.
- **first_link():** Produced a ValueError when no link was present and an AttributeError for None input.
- **shout_ratio():** Produced a ZeroDivisionError when there were no alphabetic characters and a TypeError for None input.

The initial execution resulted in three failed tests.

FAILING FUZZ TESTS

```
    +-----------------------------------
========================================== short test summary info ==========================================
FAILED test_spam_scoring.py::test_sender_domain_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_spam_scoring.py::test_first_link_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_spam_scoring.py::test_shout_ratio_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
========================================== 3 failed in 9.48s ==========================================

C:\Users\Kiran Nidhi\SE\L6_L7_33_spam_scoring_assigned\lab7>
```

7. Bug Fixes and Input Validation

The application code was modified to handle unexpected and invalid inputs.

The following changes were implemented:

- Added input type validation to all three functions.
- Added email validation to check for a valid separator and non-empty local and domain parts.
- Used find() instead of index() in first_link() to avoid exceptions when a link is not found.
- Added a check for the absence of alphabetic characters in shout_ratio() to prevent division by zero.
- Returned appropriate default values for invalid inputs.

The updated functions now handle unexpected inputs without crashing.

8. Execution of the Fixed Application

The corrected application was executed again using:

python spam_scoring.py

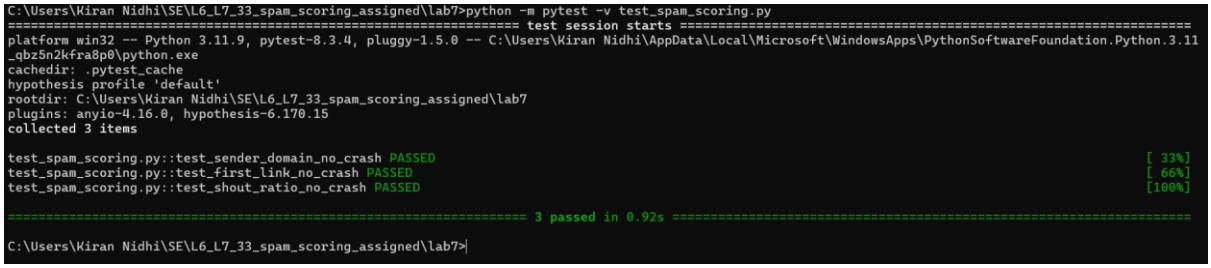
The following sample inputs were provided:

- Sender email: offers@Deals.com
- Message: Congratulations! Visit https://example.com now
- Text: FREE prize NOW

Output:

- Sender Domain: deals.com
- First Link: https://example.com
- Uppercase Ratio: 0.5833333333333334

FIXED APPLICATION EXECUTION



9. Final Fuzz Testing Results

After modifying the application, the fuzz tests were executed again using:

python -m pytest -v test_spam_scoring.py

All three tests passed successfully.

Test Case	Result
test_sender_domain_no_crash	PASSED
test_first_link_no_crash	PASSED
test_shout_ratio_no_crash	PASSED

Final Result: 3 passed in 0.92 seconds.

PASSING FUZZ TESTS

```
C:\Users\Kiran Nidhi\SE\L6_L7_33_spam_scoring_assigned\lab7>python spam_scoring.py
==== Running sender_domain ====
Enter a sender email (e.g. offers@Deals.com): offers@Deals.com
Sender Domain: deals.com

==== Running first_link ====
Enter a message containing a link: Congratulations! Visit https://example.com now
First Link: https://example.com

==== Running shout_ratio ====
Enter a message (e.g. FREE prize NOW): FREE prize NOW
Uppercase Ratio: 0.5833333333333334

C:\Users\Kiran Nidhi\SE\L6_L7_33_spam_scoring_assigned\lab7>
```

10. Conclusion

In this lab, fuzz testing was successfully implemented using the Hypothesis library in Python.

The testing process helped identify input validation bugs in the three functions of the Spam Scoring Filter application. The bugs were fixed by implementing appropriate input validation and defensive programming techniques.

After applying the fixes, all three fuzz tests passed successfully. This demonstrates how fuzz testing helps improve the reliability and robustness of software by identifying unexpected input-related failures.